

Contents

Literature Review — FinOKF: An Open Knowledge Format Profile for Equity Research with Derivation-Certified Numeric Answers	1
1. Introduction: Scope and Method of the Review	1
2. Numeric Hallucination in Financial LLMs: Error Taxonomies and Magnitudes	2
3. Verification and Certification of LLM Outputs	4
4. Verified Numeric Computation Foundations	7
5. Financial Data Representation	9
6. Format Effects and Token Economics for LLMs	12
7. Financial QA Benchmarks and Program-Based Reasoning	14
8. Synthesis: The Gap	16
9. References	19

Literature Review — FinOKF: An Open Knowledge Format Profile for Equity Research with Derivation-Certified Numeric Answers

1. Introduction: Scope and Method of the Review

This review surveys the literature bearing on a single, narrow question: what must an open, typed finance data format provide for large language model (LLM) agents to consume it with verifiable numeric fidelity — and which parts of that provision does existing work already cover? Verifiable numeric fidelity is meant precisely: every number an LLM emits when answering quantitative equity-research questions over facts carried in the format is either quoted from an identified source fact or correctly derived from declared source facts, with rounding handled soundly, and is accompanied by machine-checkable evidence of that status. The thesis this review supports proposes FinOKF, a finance profile of the Open Knowledge Format (OKF) whose bundles carry typed fact nodes and constraint blocks, together with CertiFacts, the profile’s certification layer — a claim-graph answer protocol and a deterministic checker with rounding-aware interval semantics; the profile’s typing, and the full system built on it, are evaluated causally at matched token budgets. The review is scoped to literature bearing on those components; it is not a general survey of financial natural language processing (NLP) or of hallucination at large.

Six bodies of literature meet that bar, and they organize the review. Section 2 covers empirical characterizations of numeric error in financial LLM output. Section 3 covers the young but fast-moving lineage of systems that verify or certify LLM outputs against financial source data. Section 4 covers the formal foundations a sound checker must stand on: interval and affine arithmetic, the rounding semantics of reported financial facts, XBRL (eXtensible Business Reporting Language) Calculations 1.1, and proof assistants for mechanizing soundness arguments. Sections 5–7 cover financial data representation, format effects and token economics, and financial question-answering (QA) benchmarks with their program-based baselines. Section 8 synthesizes the survey into a coverage matrix and an explicit gap analysis.

Three exclusion decisions deserve justification. First, the general hallucination literature (entity fabrication, citation invention, open-domain factuality) is excluded except where it addresses financial-numeric error specifically: the dominant recoverable error class here — scale misinterpretation over rounded, unit-laden facts — has no close analogue in open-domain factuality work. Second, approaches that improve the *model* (factuality fine-tuning, retrieval augmentation as such) appear only where they intersect output verification, because the thesis claims nothing about making models better, only about checking what they emit. Third — a point worth stating plainly in a thesis whose headline artifact is a data format — token-efficiency, context-size, and latency claims are excluded from the thesis’s motivation entirely; the format’s claimed value is typed semantics and verifiability. The format-efficiency and token-economics literature is nonetheless reviewed in §6, for two reasons: it supplies the evidence *against* a token-efficiency framing of format work, and it supplies the token-matching discipline and confound controls on which the thesis’s evaluation design rests. Measured speed-ups from prompt compression are conditional on serving configuration, largely cancel under modern inference stacks, and are unreliable through commercial serving interfaces [PromptCompression, arXiv:2604.02985]; latency therefore appears nowhere as a motivation, and token counts appear only as controlled variables.

A note on method and recency. Sources were reviewed as of July 2026. The nearest neighbors to this thesis are preprints dated between September 2025 and June 2026, most without peer review (FinGround’s ACL 2026 Industry-track acceptance is the exception). The review therefore applies a consistent hedging policy: self-reported figures from unreviewed preprints are attributed and delimited rather than treated as established; artifact unavailability is noted where encountered; and citations to classic references (interval analysis, affine arithmetic, Lean 4) were verified against publisher or arXiv records during the review.

2. Numeric Hallucination in Financial LLMs: Error Taxonomies and Magnitudes

2.1 The FAITH taxonomy and the scale-error result

Numeric failure in financial question answering is not a single phenomenon. At minimum it decomposes into fabrication — a number with no support in the source — and mangling — a number whose ingredients exist in the source but arrive corrupted in scale, sign, period, entity, or operand choice. The FAITH benchmark [FAITH, arXiv:2508.05201] provides the most useful taxonomy of these errors for financial documents and an estimate of their relative magnitudes. Its central finding is that scale errors — a value reported in thousands treated as if reported in millions, a magnitude read off a table whose unit convention lives in a header or footnote — constitute the largest recoverable error class. The headline result quantifies “recoverable”: correcting scale interpretation alone lifts Llama-3.3-70B from 37% to 57.7% accuracy on FAITH’s evaluation — a 20.7-percentage-point gain from repairing a single error class, without touching the model’s arithmetic or reasoning.

The natural reading is that much financial-numeric failure is not arithmetic incapacity but misreading of reporting conventions. Financial tables externalize scale: a cell contains “391,035”,

and the fact that this denotes roughly \$391 billion is carried by a header (“in millions”) possibly hundreds of tokens away, or by domain convention alone; any pipeline that linearizes such tables into prose or key-value text inherits this fragility. This is the strongest empirical motivation available for a *typed* fact representation in which scale, unit, currency, and period are explicit fields of each fact rather than ambient context.

The result must nonetheless be read with care. FAITH shows that fixing scale interpretation recovers accuracy within its own pipeline; it does not show that a typed input representation causally reduces scale errors against an untyped rendering of the same facts at a matched token budget. Typed renderings are longer than terse tables, and longer inputs — or accompanying prose documentation — are confounds in their own right (§6 returns to the documentation confound). On the evidence reviewed here, the causal question remains open, which is why the thesis treats it as a hypothesis to be tested rather than a premise.

2.2 The failure of post-hoc LLM verification

If numbers are unreliable, the obvious industrial remedy is to add a second model as a verifier. FinVerBench [FinVerBench, arXiv:2605.29586] measured how well that remedy works for financial claims, and the answer is: essentially not at all. LLM verifiers in its evaluation exhibit false-positive rates of 95–100% — presented with incorrect financial claims, they endorse nearly all of them. A verification stage that accepts practically every wrong claim contributes approximately zero evidential value on exactly the failure class it exists to catch, while manufacturing unearned confidence in “verified” answers.

FinVerBench’s rule-based verifiers show the mirror-image profile: they do not hallucinate approval, but they hit a recall ceiling of roughly 52% — about half of true statements fall outside what the hand-built rule set can confirm. The pairing is more instructive than either result alone: LLM judging fails on soundness, rule-based checking is sound-leaning but incomplete, and together they dictate the honest form deterministic verification must take.

2.3 Why LLM judging fails, and what follows for system design

Several mechanisms plausibly explain the verifier false-positive rates, though none is conclusively established. Generator and judge share training distributions and failure modes: a mangled number of plausible magnitude is precisely what a generator tends to produce and what a judge finds plausible. Verifying a *derived* value — a margin, a growth rate — requires re-deriving it, so the judge must perform the same arithmetic the generator got wrong, with the same machinery. And LLM-as-judge protocols are sensitive to surface properties of answers, such as verbosity — a sensitivity to which LLM-judge protocols in adjacent financial-QA evaluation are exposed (§7.2) — whereas a deterministic computation is immune to that confound by construction. Whatever the mechanism, the FinVerBench false-positive rates stand independently of any explanation of them.

Taken jointly, FAITH and FinVerBench impose three design constraints on any system claiming trustworthy financial numbers. First, the recoverable core of the problem is interpretation of scale, unit, and period, so explicit typing of those attributes is the natural lever. Second, the checking mechanism must be deterministic, non-LLM code. Third, deterministic checking carries a completeness ceiling, so coverage — the fraction of true answers the checker can actually confirm

— must be measured rather than assumed, and statements outside coverage must fail closed (be explicitly uncertified) rather than pass silently. The lineage reviewed next can be read as a series of partial responses to these constraints.

3. Verification and Certification of LLM Outputs

The verification-adjacent lineage in financial NLP has compounded quickly: an October-2025 benchmark for auditing XBRL filings was followed by FinGround (April 2026), FinVerBench (May 2026), and AuditFlow (June 2026), with Proof-Carrying Numbers (September 2025) standing slightly apart as a theory-first contribution. This section reviews the four works closest to the thesis, ordered by what they verify — answers post hoc, generation inputs, filings, individual emitted numbers — and closes with a lineage table making each work’s gap explicit.

3.1 Post-hoc recomputation: FinGround

FinGround [FinGround, arXiv:2604.23588] is the most direct precedent for checking LLM answers by recomputation. It verifies financial answers post hoc using 47 hand-written recomputation templates: when a claimed value matches a known calculation pattern, the corresponding template recomputes it from source data and flags disagreement. The approach is deterministic, answer-level, and practical at research-benchmark scale: the template library covers roughly 80% of FinGround’s second evaluation set (which its authors call D2).

Its limits are structural rather than incidental. Coverage is achieved by enumeration — 47 patterns — rather than by composition: a derivation outside the template library cannot be checked at all, and each new calculation type costs another hand-written template. Verification is performed *on* the answer rather than *carried by* the answer: nothing in the protocol, as described, requires the model to disclose which facts it used or what it computed, so recomputation must reconstruct the intended derivation. And reproducibility is currently compromised: the paper’s code repository returned a 404 error at review time (July 2026), so any experimental comparison against FinGround requires budgeting a reimplementation. FinGround establishes the practical value of deterministic recomputation; it does not provide answer-borne certificates, compositional coverage, or a stated soundness guarantee.

3.2 Typed fact ledgers without a checker: VeNRA

VeNRA [VeNRA, arXiv:2603.04663] approaches the problem from the input side. It introduces a typed “Universal Fact Ledger” — a structured inventory of numeric facts against which generation is grounded — and is, to this review’s knowledge, the closest existing commitment to *typed numeric facts* as a substrate for financial LLM work. VeNRA reports a 1.2% hallucination rate for its grounded configuration; that figure is measured under the authors’ own metric, in an unreviewed preprint, and has not been independently reproduced, so this review cites it as the authors’ claim rather than an established property.

The instructive gap is architectural. VeNRA’s ledger types facts but has no constraint or derivation layer: nothing validates relationships between facts (subtotal consistency, period alignment), and nothing checks what a model *computes from* the facts. A model that quotes real ledger entries and combines them into a wrong ratio produces output that is fully “grounded” by VeNRA’s lights and numerically false. Typing addresses the interpretation problem of §2.1 but not the verification

problem of §2.2: a typed substrate without a checker bounds neither fabrication of derived values nor arithmetic error over genuine facts.

3.3 Agentic symbolic audit of filings: AuditFlow

AuditFlow [AuditFlow, arXiv:2606.03031] verifies a different object altogether: the filing itself. It builds an agentic environment in which an LLM audits XBRL filings using symbolic tools, and its key evidential contribution is an ablation: performance collapses from 82% to 18% when the symbolic tools are removed. This is the most direct evidence in the lineage that the verification capability lives in the symbolic layer — deterministic computation over structured facts — rather than in the LLM, which functions as an orchestrator. The ablation corroborates the FinVerBench lesson from the opposite direction: where FinVerBench shows LLM judgment failing, AuditFlow shows symbolic tooling succeeding.

The scope limitation is equally clear: AuditFlow audits filings for internal consistency and says nothing about whether an LLM’s *answer about* a filing is correct — a perfectly consistent filing coexists with an assistant that mangles every number quoted from it. AuditFlow’s baseline comparison figures beyond the ablation were not re-verified against the primary source and are not relied on here.

3.4 Fail-closed certification of atomic values: Proof-Carrying Numbers

Proof-Carrying Numbers (PCN) [PCN, arXiv:2509.06902], from World Bank researchers, is the theoretical anchor of this lineage and the work this thesis’s certification layer most directly extends, so it warrants depth. PCN proposes a per-number certification protocol: each numeric value an LLM emits must carry a certificate binding it to an identified source fact; a deterministic checker validates certificates; and presentation is *fail-closed* — a number whose certificate does not validate is never presented as verified. The guarantees are stated as theorems: the paper’s Theorems 5.1 and 5.3 establish soundness — under the protocol’s assumptions, the fail-closed property holds and unsupported numbers cannot acquire verified status. This is a different kind of claim from anything else in the lineage: not an empirical error rate, but a provable property of a checking protocol.

Three delimitations define both PCN’s current standing and the opening it leaves. First, and most importantly, the protocol certifies *atomic quoted values only*: it can certify that “revenue was \$391,035 million” faithfully quotes a source fact, but the derived quantities that dominate analyst practice — margins, growth rates, ratios — lie outside the protocol and are explicitly declared future work by its authors. The extension is not incremental: a quoted value can be checked by identity, whereas a value derived from rounded operands is only checkable up to a tolerance (§4.3), which demands an error model for what fabrications the tolerance admits and a soundness argument that survives the extension. Second, the paper contains no experiments: feasibility on real data, false-reject behavior, and model compliance with the emission protocol are all unmeasured. Third, its reception is so far nil — zero citations as of mid-2026 — so the protocol has not been stress-tested, extended, or independently implemented. None of these is a flaw in what PCN proves; together they mark exactly where the certified-derivations gap sits.

3.5 The lineage at a glance

Table 1 summarizes the works of this section against the design constraints derived in §2.3.

Work	Object verified	Mechanism	Guarantee offered	Gap relative to derivation-certified answers
PCN [PCN, arXiv:2509.06902]	Individual numbers in LLM answers	Per-number certificates; fail-closed deterministic checker	Soundness theorems (Thms. 5.1, 5.3)	Atomic quotation only; derivations declared future work; no experiments
FinGround [FinGround, arXiv:2604.23588]	LLM answers, post hoc	47 hand-written recomputation templates	Empirical; ~80% coverage of its second evaluation set	Enumerated, non-compositional coverage; no answer-borne certificate; code unavailable
VeNRA [VeNRA, arXiv:2603.04668]	Iteration grounding	Typed Universal Fact Ledger	Self-reported 1.2% hallucination rate (authors’ metric; not reproduced)	No constraint or derivation layer; no checker; derived errors pass unchecked
AuditFlow [AuditFlow, arXiv:2606.03031]	XBRL filings	Agentic LLM over symbolic audit tools	Empirical; 82%→18% ablation without symbolic tools	Verifies filings, not model answers
FinVerBench [FinVerBench, arXiv:2605.29586]	Verifiers themselves	Benchmark of LLM and rule-based verifiers	Measurement, not a mechanism	Diagnoses failure (95–100% verifier false positives; ~52% rule recall ceiling); builds nothing

Reading down the gap column: each work contributes one ingredient — typed facts (VeNRA), deterministic recomputation (FinGround), symbolic tooling with measured value (AuditFlow), provable fail-closed certification (PCN) — and none combines certification of *derived* values with a soundness guarantee and empirical evaluation. That empty cell is the target of this thesis’s certification layer, CertiFacts. The tempo of the lineage — five substantial entries between September

2025 (PCN) and June 2026 (AuditFlow), roughly nine months — also indicates the gap is visible to the field and should not be assumed to stay open.

4. Verified Numeric Computation Foundations

Certifying a derived financial value requires machinery the NLP literature of §3 does not supply: enclosure semantics for arithmetic over imprecisely known operands (§4.1–§4.2), an account of *why* reported facts are imprecisely known (§4.3), engagement with the production standard that already checks financial arithmetic under rounding (§4.4), and a means of making the checker’s own guarantee trustworthy (§4.5).

4.1 Interval arithmetic and the dependency problem

Interval arithmetic, systematized in the standard reference by Moore, Kearfott, and Cloud [IntervalAnalysis, SIAM 2009], replaces each uncertain quantity with an interval guaranteed to contain it, and defines arithmetic so that enclosure is preserved: evaluating an expression with interval operands yields an interval containing every value the expression can take as the operands range over their intervals. For a certification checker, enclosure does one direction of the work for free: if a claimed value is genuinely consistent with the reported facts, the true derived range lies inside the computed interval, so a checker that certifies whenever the claim’s rounding envelope intersects that interval never wrongly rejects a consistent claim through under-coverage.

The cost is paid in the other direction, and this is where naive interval arithmetic fails the certification use case. Interval evaluation treats every operand occurrence as independent, so correlations between repeated operands are lost — the *dependency problem*. The canonical example is $x - x$: evaluated over $[\underline{x}, \bar{x}]$, it yields $[\underline{x} - \bar{x}, \bar{x} - \underline{x}]$ rather than $[0, 0]$. The problem is not exotic in finance; it appears in the most ordinary derivation there is. A gross margin has the form $(f_1 - f_2)/f_1$, with f_1 occurring twice; naive interval evaluation lets the two occurrences vary independently, producing an interval strictly wider than the true range. In ordinary numerical analysis, excess width is mere imprecision. In certification it is the *false-accept surface*: every unit of unnecessary width admits additional wrong values whose envelopes now intersect the computed interval, and the effect compounds with derivation depth. A checker built on naive intervals is thus sound in its rejections but systematically too permissive in its acceptances. Mitigations of two kinds are available. Symbolic simplification before evaluation and monotonicity analysis are standard instruments of the self-validated-computation literature [IntervalAnalysis, SIAM 2009]. The further strategy this thesis adopts — exact arithmetic over the underlying representations, with intervals introduced solely at the uncertainty boundaries — is chosen because the financial setting specifically permits it: operand uncertainty there enters only at discrete rounding points, for reasons §4.3 makes precise.

4.2 Affine arithmetic

Affine arithmetic [AffineArithmetic, Numerical Algorithms 2004] is the classical self-validated remedy for the dependency problem. Each quantity is represented as an affine combination of noise symbols, $x = x_0 + x_1\varepsilon_1 + \dots + x_n\varepsilon_n$ with each $\varepsilon_i \in [-1, 1]$, and — crucially — the *same* noise symbol represents the same uncertainty source wherever it appears. First-order correlations are thereby preserved: $x - x$ evaluates to exactly zero, and expressions with repeated operands

stay tight where plain intervals balloon. The trade-offs are equally classical: affine forms are exact only for affine operations, while nonlinear operations such as multiplication and division require approximation and fresh noise symbols. Common financial derivations occupy a convenient middle ground — differences and rescalings are affine, and ratios typically introduce a single division — so affine forms are a natural instrument for whatever operand dependency survives exact-rational evaluation.

4.3 Rounding semantics of reported financial facts

Why are reported facts imprecisely known at all? Because financial reports round. In XBRL, every numeric fact carries a `decimals` attribute that makes the rounding explicit: `decimals = d` asserts that the reported value is accurate to 10^{-d} , with negative d moving the accuracy boundary left of the decimal point — $d = -6$ means rounded to millions [XBRL21, XBRL 2.1 spec]. The denotational reading, adopted by the thesis and — as §4.4 shows — already standard in filing validation, is that a reported value v with `decimals` d denotes not a point but the interval of true values that round to it: $[v - \frac{1}{2} \cdot 10^{-d}, v + \frac{1}{2} \cdot 10^{-d}]$. Apple’s FY2024 revenue, reported as 391,035 with $d = -6$ (US dollars in millions), denotes a true value anywhere in $[391,034.5, 391,035.5]$ million.

Two consequences follow, and both are semantic rather than implementational. First, a value derived from rounded facts has no single correct answer to compare against: a gross margin computed from rounded revenue and rounded cost of sales is pinned down only to the interval induced by the operand envelopes, so a checker that recomputes “the” value and tests exact equality commits a category error — and will produce false rejects on perfectly correct answers. Second, the claimed answer is itself rounded (a margin stated as 46.2% asserts three-decimal precision), so consistency checking is inherently two-sided: certification must mean that the claim’s own rounding envelope intersects the interval derived from the operands’ envelopes. The thesis’s running worked example makes the semantics concrete. Apple’s FY2024 cost of sales, reported as 210,352 (also $d = -6$), denotes $[210,351.5, 210,352.5]$ million; exact-rational evaluation of the gross-margin program $(f_1 - f_2)/f_1$ over the two envelopes yields a tight interval around 0.46206; the claimed 46.2%, whose own three-decimal envelope is 0.462 ± 0.0005 , intersects that interval and certifies, whereas a fabricated 48.1% — or a claim citing a nonexistent fact identifier — fails closed. A structural observation connects back to §4.1: operand uncertainty here enters *only* at rounding boundaries, because the reported mantissas are exact integers — precisely the regime in which exact rational arithmetic over mantissas, with envelopes applied only at the boundaries, sidesteps most of the dependency problem rather than fighting it.

4.4 XBRL Calculations 1.1: rounding-aware validation within a filing

The interval reading of `decimals` is not a novelty; it is production practice. XBRL Calculations 1.1, an XBRL International Recommendation of February 2023 [XBRLCalc11, XBRL International 2023], upgrades the validation of calculation-linkbase summations — filer-declared assertions of the form “these child line items sum to this subtotal” — to be rounding-aware: rather than flagging every summation that fails exact arithmetic, it derives intervals from the facts’ `decimals` attributes and checks whether the summation *could* hold given the declared roundings. Its existence matters for two reasons. Practically, it validates the interval semantics of §4.3 at regulatory scale.

Positionally, it is the closest formal precedent to the CertiFacts checker, so the differentiation must be exact. Calculations 1.1 checks *within-filing linkbase summations*: fixed additive relationships among facts inside one filing. It does not address arbitrary arithmetic (ratios, growth rates, per-share quantities), derivations that cross periods or filings, or — self-evidently — anything emitted by an LLM. Its implicit adversary is honest rounding noise, not fabrication; accordingly it neither needs nor offers fail-closed presentation semantics, an error model bounding what a tolerance admits, or a soundness theorem. A derivation-certification checker inherits Calculations 1.1’s rounding semantics and departs from it in object (LLM answers rather than filings), scope (arbitrary declared derivations rather than fixed summations), and guarantee (a proved fail-closed property with an explicit false-accept error model).

4.5 Mechanized soundness: Lean 4 and mathlib

The final foundation concerns trust in the checker itself. Lean 4 [Lean4, CADE 2021] is an interactive theorem prover and full programming language, reimplemented in itself and extensible from parser to tactic framework; mathlib [mathlib, arXiv:1910.09336] is its community-built library of formalized mathematics, providing among much else the rational-number arithmetic and order-theoretic infrastructure needed to state and prove interval-evaluation lemmas over exact rationals.

Mechanization is proportionate here rather than ornamental. The soundness claim of a certification checker — fabricated numbers never certify — is a small, self-contained algebraic theorem about interval evaluation, but exactly the kind of statement where paper proofs quietly fail: boundary and tie cases of half-unit envelopes, sign handling under negative decimals, the interaction of two-sided tolerances. A system whose entire value proposition is a guarantee is worth only as much as that guarantee is reliable, and the only comparable guarantee in the §3 lineage — PCN’s soundness theorems — is pen-and-paper. Mechanizing a derivation-level soundness theorem therefore both extends the PCN result and strengthens its assurance in the same step. The realistic scope should be stated honestly: the fail-closed soundness theorem and core tolerance-algebra lemmas are tractable formalization targets, whereas probabilistic false-accept bounds under a fabrication model rest on distributional assumptions better handled on paper. On the evidence gathered for this review, no published work mechanizes a soundness proof for an LLM-output certification checker; the combination is, at the time of writing, untried.

5. Financial Data Representation

Sections 2–4 established the problem (numeric hallucination) and the verification machinery (deterministic recomputation, interval semantics, mechanized soundness). Both presuppose something easy to take for granted: a machine-readable representation of the source facts against which emitted numbers can be checked. This section reviews the representations actually available for financial reporting data — the regulatory tagging standard (XBRL), the open container on which the FinOKF profile is built (OKF), and the standards-track ontology family (FIBO) — and closes with the argument that representation, however rich, is not verification.

5.1 XBRL and iXBRL: typed numeric facts at regulatory scale

XBRL is the tagging standard under which US public companies file with the SEC (Securities and Exchange Commission); Inline XBRL (iXBRL) embeds the same tags in the human-readable HTML

(Hypertext Markup Language) filing. An XBRL fact carries a taxonomy concept, a numeric value, a unit, a context binding the value to an entity and a period, and a decimals attribute recording the rounding applied to the reported figure, with the interval semantics set out in §4.3 [XBRL21, XBRL 2.1 spec]. This is, on paper, exactly the payload a typed fact format needs: typed, entity- and period-scoped, unit-bearing, rounding-annotated numeric facts, produced at regulatory scale and available at no cost from the SEC’s EDGAR (Electronic Data Gathering, Analysis, and Retrieval) system.

The scale of the concept vocabulary is the first complication. The US-GAAP (United States Generally Accepted Accounting Principles) 2024 taxonomy contains 17,388 concepts [FinTagging, arXiv:2505.20650]. A representation whose semantics depend on correctly selecting one concept out of seventeen thousand is only as usable as the alignment step that does the selecting — and the evidence on that step is sobering.

5.2 The concept-alignment bottleneck: FinTagging

FinTagging [FinTagging, arXiv:2505.20650] is the most informative study here because it decomposes XBRL tagging into two subtasks and measures them separately: FinNI (identifying and extracting numeric facts from filing text and tables) and FinCL (linking each extracted fact to its taxonomy concept). The asymmetry between the two is large. DeepSeek-V3 reaches an extraction F1 of 0.7238 on FinNI, but only 0.1715 accuracy on FinCL concept linking. When the linking task is posed as direct classification over the full 17,388-concept space, both GPT-4o and DeepSeek-V3 score 0.0 — not low, zero. The benchmark itself is well-grounded: 30 FY2024 10-K filings, 81,325 facts, with gold labels taken from the issuers’ own XBRL tags, so the supervision is free and authoritative rather than annotated post hoc. One methodological detail deserves note: FinTagging linearizes filing tables into Markdown key-value text (averaging roughly 999.7 tokens per table), which is itself a rendering decision of the kind §6 shows can carry measurable effects; the reported numbers are therefore relative to one particular rendering.

Two consequences follow for this thesis. First, extraction is close to tractable while concept semantics is the bottleneck; any pipeline in which an LLM performs taxonomy alignment inherits a failure rate that no downstream checking can repair, because the checker would be checking against wrongly-labelled facts. Deterministic bundle construction from the issuers’ own tags sidesteps the bottleneck entirely: the alignment was already performed by the filer, under regulatory obligation. Second, the FinCL numbers are a caution against any design in which the model must *name* concepts at answer time; in the CertiFacts claim-graph protocol the model only ever references opaque fact identifiers, never taxonomy concepts.

5.3 Extension tags and the limits of taxonomy retrieval

Of FinTagging’s 81,325 facts, 11,874 — roughly one in seven — carry company-specific *extension* tags: concepts the issuer defined because no US-GAAP concept fit [FinTagging, arXiv:2505.20650]. Extension tags are structurally unaddressable by any method that retrieves candidate concepts from the standard taxonomy, since the concept does not exist there. This has two design implications. For representation: a fact format built on a closed vocabulary will under-cover real filings; the format must be able to carry issuer-defined concepts verbatim, with provenance, rather than

forcing normalization into the standard taxonomy. For certification: derivations that combine an extension concept from one firm with a standard concept from another are not certifiable as *comparable* without assumptions the data does not license; within-firm derivations over extension facts remain checkable, because the arithmetic and rounding semantics do not depend on the concept’s provenance. This is a scoping fact worth stating plainly rather than an engineering detail to be worked around.

5.4 The OKF container, honestly characterized

The second representation layer reviewed — and the container this thesis profiles — is Google’s draft OKF specification (`okf/SPEC.md` in the `GoogleCloudPlatform/knowledge-catalog` repository) [OKF, github.com/GoogleCloudPlatform/knowledge-catalog]. The spec is a v0.1 draft, Apache-2.0 licensed, and explicitly “not an official Google product”. Its design is deliberately minimal: a bundle is a directory of Markdown files with YAML (YAML Ain’t Markup Language) frontmatter; concept identity is the file path; `index.md` and `log.md` are reserved; the only *required* frontmatter field is a free-string type; unknown frontmatter keys must be preserved by conforming tools, which — together with custom type values and fenced payload blocks — constitutes an extension mechanism through which strictly conformant domain profiles can be defined. Links between files are untyped edges. The spec states an explicit non-goal of “replacing domain-specific schemas (Avro, Protobuf, OpenAPI) — OKF references them; it does not subsume them”.

Equally important is what the specification does not contain. It has no numeric types, no units, no periods, and no validation semantics of any kind. In finance these are exactly the absences that matter most: §2.1 established that scale and unit misinterpretation over rounded, convention-laden facts is the largest recoverable error class, and a container carrying no unit, scale, period, or rounding semantics leaves every one of those hazards to the consumer. Its text contains no occurrence of “token”, “latency”, “context window”, “retrieval”, “RAG”, “embedding”, “chunk”, or “finance”. Any characterization of OKF as token-efficient, LLM-optimized, or finance-aware therefore has no basis in the spec; claims of that kind, which circulate informally, should be treated as projection. Governance is also thin: the repository carries roughly 55 open issues with the maintainers largely silent, and the dominant community conversation concerns the format’s “trust surface” rather than its data model. The honest summary is that OKF supplies packaging conventions, an extension mechanism admitting strictly conformant domain profiles, and a possible interoperability and community timestamp — nothing more. All numeric, unit, period, and constraint semantics in a finance profile must be supplied by the profile itself. That is exactly the layer FinOKF fills: an OKF-conformant profile supplying the typed numeric semantics — unit, scale, currency, period, rounding, and constraint blocks — that the container deliberately leaves open, semantics strong enough (§4) to support deterministic certification of derived numbers through the CertiFacts protocol and checker. OKF packages the FinOKF bundle; it is never the mechanism of any claimed effect.

5.5 FIBO

For completeness, the standards-track ontology family deserves a paragraph. FIBO (the Financial Industry Business Ontology), hosted by the EDM Council and standardized through the OMG (Object Management Group), is expressed in OWL (Web Ontology Language) and “defines the sets

of things that are of interest in financial business applications and the ways that those things can relate to one another” [FIBO, spec.edmcouncil.org/fibo]. FIBO operates at the level of terminology — entities, instruments, roles, and their relations — and its Description Logic foundation supports classification and conceptual-consistency reasoning, not arithmetic over reported rounded values. It has no counterpart to XBRL’s `decimals` semantics and no notion of answer verification. It is complementary to the present work (a future revision of the FinOKF profile could borrow FIBO identifiers for concept typing) but addresses neither of the two problems this thesis is about: representing reported numeric facts with their rounding semantics, and certifying numbers derived from them.

5.6 Representation is not verification

The recurring temptation in this literature is to treat richer representation as if it were verification. Two observations argue against this. First, VeNRA (§3.2) fields exactly such a substrate — a typed fact ledger much like this thesis’s FinOKF bundle — yet provides no constraint or derivation layer: typing gave the facts structure, and nothing enforced that emitted numbers respect it. Second, XBRL has provided the richest typed numeric representation in any applied domain for two decades, and numeric hallucination in financial LLMs persists at the rates documented in §2. The reason is simple: representation lives on the input side of the model; hallucination happens at emission. A typed representation is *necessary* for deterministic checking — it gives the checker identities to bind to and carries the `decimals` semantics without which tolerance checking is not well-defined — but not *sufficient*. Sufficiency requires an emission protocol forcing the model to declare, for every number, where it came from or how it was computed, plus a checker that re-executes those declarations. That is the separation this thesis draws between FinOKF (the format) and CertiFacts (the protocol and checker), and why §8 scores representation and verification in separate columns.

6. Format Effects and Token Economics for LLMs

The thesis makes a causal claim: that typing scale, unit, and period information reduces numeric errors *at matched token budgets*. The literature reviewed in this section is where the methodological discipline behind that phrase comes from — and where several tempting claims a format thesis might otherwise lean on (compression headlines, latency wins, structure effects) are shown to dissolve under controls.

6.1 TOON and the limits of compression claims

TOON (Token-Oriented Object Notation) [TOON, github.com/toon-format/toon] is a compact encoding of the JSON (JavaScript Object Notation) data model designed to reduce token counts in LLM prompts. Its README reports 40–59% token reductions on uniform arrays under the `o200k_base` tokenizer. The same benchmark suite, read closely, bounds the claim sharply: TOON *loses* to CSV (comma-separated values) by 6–9% on flat tables — the structure most common in financial statements — and compact JSON beats TOON by roughly 14.7% on nested structures. On the project’s own accuracy benchmark TOON and JSON are approximately tied (TOON 76.4 versus JSON 75.0, per the README’s retrieval-accuracy table). Compression is thus regime-dependent, and downstream accuracy is not obviously improved anywhere. These results were

among the reasons the present thesis — although its headline artifact is a data format — was scoped *away* from token-efficiency as a contribution: the honest reading of the evidence is that token counts are a variable to be *controlled*, not a prize to be won, and FinOKF’s claimed value accordingly lies in typed semantics and verifiability, not compression.

6.2 Generation is not comprehension: the emission penalty and the prompt tax

A 21-model study of format generation [TOONGen, arXiv:2603.03306] adds the output-side half of the picture, and it is decisive for protocol design. Three findings matter. First, the “prompt tax”: teaching a model an unfamiliar format costs instruction tokens — in one measured case an additional 70% — so compression accounting that ignores the teaching overhead overstates the benefit. Second, generation of exotic notations fails in ways comprehension does not: TOON achieved 0% one-shot validity on deep-nesting and invoice-generation tasks, while plain JSON under structured-output constraints was the best performer on simple tabular generation (556 tokens, 92.9% one-shot). Across formats, the study finds a generation penalty of roughly 9–14 percentage points for exotic notations relative to JSON, a figure corroborated independently [FormatCost, arXiv:2605.29676]. Third, constrained decoding — the natural tool for forcing valid structured output — can itself degrade quality catastrophically: Hermes-4-405B fell from 92.5 to 35.0 under constrained decoding in one condition.

The design consequences for the CertiFacts protocol are direct. Reading a format and writing a format are different capabilities, and the asymmetry favors conservative output design: the claim-graph the model emits is plain JSON — the format with the strongest generation reliability in the studies reviewed here — with no custom syntax anywhere on the output side. And because even JSON-constrained emission carries a measurable risk, the constraint tax is treated as an empirical quantity to be measured (RQ4/G4; RQ1–RQ4 are the thesis proposal’s numbered research questions, each mapped one-to-one to a pre-registered kill-gate G1–G4 that decides whether the corresponding line of work proceeds), with intermittent constraining held as a fallback, rather than assumed benign.

6.3 SECQUE and TQA-Bench: what format studies measure, and what they omit

Two studies bracket the evaluation design space. SECQUE [SECQUE, arXiv:2504.04596] studies format and token effects on question answering over SEC filings — the exact domain of this thesis. What it measures, however, is answer accuracy under format variation and no verifiability outcome of any kind: it can say which rendering helps a model answer, and nothing about whether any answer’s numbers can be certified. TQA-Bench [TQA-Bench, arXiv:2411.19504] contributes a multi-scale tabular question-answering design — the same content evaluated at varied context scales — which this thesis borrows as an evaluation template: holding content fixed while varying budget is the precursor of the stricter discipline of matching budgets across representational arms. Neither work types facts, certifies derivations, or measures gaming; they are methodology sources, not competitors.

6.4 The documentation confound

The most underappreciated threat to validity in this literature comes from the semantic-layer study [SemanticLayer, arXiv:2604.25149]: when “semantic layer” structure was added to

LLM analytics pipelines, prose documentation explained nearly all of the measured gains. The structure and the words describing it arrive together, and the words were doing the work. This generalizes to any typed-versus-untyped comparison, including this thesis’s central one: a typed fact format normally carries names, labels, and descriptions that an untyped rendering lacks, so a naive comparison confounds structure with documentation text. The G2 experiment therefore uses a factorial design — {typed, untyped} × {with, without prose documentation} at matched token budgets — as a mandatory control; without it, a positive typed-format result would be uninterpretable.

6.5 The latency reality check, and why this thesis excludes latency claims

Finally, the largest-scale study of prompt-compression serving behavior [PromptCompression, arXiv:2604.02985] — over 30,000 runs — explains why no latency claim appears anywhere in this thesis. Speed-ups above 1.3× materialize only under naive serving stacks, or for prompts above roughly 10k tokens compressed at better than 4× ratios. Under vLLM, gains at realistic scales cancel or invert: 0.4–0.9× at prompts of 4k tokens or fewer. Against commercial APIs (application programming interfaces), no reliable speed-up was observed at all. The paper’s headline “up to 18%” figure holds only in a matched operating window, and the one robust benefit found is a reduction of up to 75% in peak GPU (graphics processing unit) memory — an infrastructure win, not a user-facing latency win. Token counts still matter for API cost and context budget, but the inference from “fewer tokens” to “faster answers” does not survive modern serving. Accordingly, token matching in this thesis’s evaluation is a causal control — it removes budget as a confound between arms — and never an efficiency claim. Concretely, matching means that every arm renders identical fact content, with renderings padded or trimmed to equal measured token counts under two pinned tokenizers (o200k_base and the Llama tokenizer), and counts are reported under both. Latency appears in this thesis only as this caveat.

7. Financial QA Benchmarks and Program-Based Reasoning

7.1 The FinQA lineage

The core numeric-reasoning benchmarks form a clear lineage. FinQA [FinQA, arXiv:2109.00122] contributed expert-written questions over financial reports annotated with gold *programs* — small arithmetic derivations over values in the source — which made derivations first-class evaluation objects. Those program annotations are also why FinQA is convertible, programmatically, into claim-graph gold for this thesis (dataset D2), though the conversion pipeline and its 100-item audit are budgeted as real work rather than assumed free. ConvFinQA [ConvFinQA, arXiv:2210.03849] extends the setting to conversational multi-turn reasoning, where numbers computed in one turn feed later turns — a natural stress test for provenance chains. TAT-QA [TAT-QA, arXiv:2105.07624] covers hybrid table-plus-text evidence with arithmetic-heavy questions. What the lineage shares is its scoring model: final answers are matched against gold, sometimes with tolerance, and the *derivation* the model actually used is unexamined at evaluation time. A model that reaches the right number through a wrong or fabricated intermediate is scored identically to one that derived it correctly — precisely the blind spot certification targets.

7.2 FinanceBench and FinDER: realistic queries, fragile pipelines

FinanceBench [FinanceBench, arXiv:2311.11944] moved to open-book question answering over full filings, trading the FinQA lineage’s clean program annotations for realism. FinDER [FinDER, arXiv:2504.15800] is the most instructive recent entry: 5,703 expert-constructed query–evidence–answer triplets over 490 S&P 500 10-K filings (as of 2024-12-31), with queries written in the terse, jargon-laden style analysts actually use, distributed under CC-BY-NC-4.0. Its results carry two lessons this thesis is built around. First, the oracle-context lesson: retrieval recall over FinDER is very low (Context Recall of 11.68 for BM25, rising only to 25.95 for E5-mistral), and Response Correctness is 59–68 with oracle context versus 28–34 with top-10 retrieval. Retrieval noise dominates every downstream effect; representation and verification effects are only cleanly measurable when context is supplied, not retrieved — so this thesis’s evaluation supplies FinOKF bundles as given context and excludes retrieval from all causal claims. Second, a preprocessing lesson: FinDER’s shipped evidence consists of plain-text paragraphs in which the original tables were destroyed during corpus construction, foreclosing exactly the structured-versus-unstructured comparisons one would want to run on it; structured evaluation arms must be rebuilt from EDGAR primary data. A final caution: FinDER evaluates with RAGAS, an LLM-judge evaluation framework, whose protocol is exposed to a verbosity confound; deterministic gold-aligned scoring — in which every numeric token in an answer is aligned to a gold value under rounding tolerance and classified by error type, identically in every evaluation arm, as this thesis’s symmetric primary metric does — is immune to it.

7.3 Program-of-Thoughts: the strong-baseline family

Chain-of-Thought prompting [CoT, arXiv:2201.11903] established that eliciting intermediate reasoning steps improves arithmetic and symbolic reasoning, but the computation still happens inside the model, so arithmetic slips survive. Program-of-Thoughts (PoT) [PoT, arXiv:2211.12588] made the decisive move for numeric work: the model *writes a program* expressing the reasoning, and execution is delegated to an external interpreter, eliminating in-model arithmetic error. PoT over a CSV rendering of the facts is therefore the honest strong baseline for this thesis (RQ3/G3): it already enjoys deterministic computation, so any advantage of the full FinOKF + CertiFacts arm must come from something other than calculator access.

What PoT does not provide is equally precise. Its program variables are whatever the model extracted from context: an operand read at the wrong scale, taken from the wrong period, or fabricated outright is executed flawlessly, and nothing in the protocol detects it. The FAITH finding (§2.1) that scale misinterpretation is the largest recoverable error class [FAITH, arXiv:2508.05201] describes exactly the error family that execution does not fix — a program computing over a figure read in thousands instead of millions is wrong at full machine precision. PoT also yields no consumer-facing artifact: there is no certificate, no fail-closed behavior, and no way for a downstream user to distinguish a grounded program from a confabulated one. The G3 comparison is designed so that if PoT-over-CSV matches the full FinOKF + CertiFacts arm at equal tokens on verified numeric accuracy — accuracy under the gold-aligned symmetric metric described in §7.2 — the thesis’s residual contribution reduces, explicitly and by prior commitment, to certification coverage — a narrower but still defensible claim.

7.4 Constrained decoding

Constrained decoding — restricting generation to a grammar or JSON schema — is the mechanism by which claim-graph emission can be enforced, and it completes the strong-baseline family because it guarantees syntactic validity of structured outputs. The literature imposes two caveats. First, the quality risk documented in §6.2 — a 92.5→35.0 collapse in one measured condition — means enforcement is not free; the thesis measures this constraint tax directly and holds intermittent constraining as the fallback. Second, syntactic validity is not semantic validity: a schema-valid claim-graph can still reference a nonexistent fact identifier, mix periods, or assert an arithmetic falsehood. Constrained decoding narrows the output space; only the deterministic checker provides guarantees about what the output *means*. It is infrastructure for certification, not a substitute for it.

8. Synthesis: The Gap

8.1 Coverage matrix

The gap this thesis occupies is, stated compactly, the empty combination of four things: an open container, typed numeric semantics, derivation certification, and causal evaluation. No reviewed work supplies all four. Table 2 refines the measurable part of that statement into a matrix: the open-container element is a packaging property argued in §5.4 rather than a scored column, while the certification and evaluation elements are split into finer properties — certification of derived values, its fail-closed guarantee, token-matched causal evaluation, and gaming resistance. Rows are the systems and studies reviewed (§§2–7); columns are the five measurable properties whose conjunction defines the FinOKF + CertiFacts proposal: **typed facts** (does the system represent numeric facts with concept, unit, scale, period, and rounding metadata?), **derived-value certification** (does it check numbers *computed from* facts, not merely quoted?), **fail-closed soundness** (is there a guarantee that unverifiable numbers are rejected rather than passed?), **token-matched causal evaluation** (are representational effects measured with token budgets controlled across arms, matched in the sense defined in §6.5?), and **gaming evaluation** (is the verification mechanism itself adversarially attacked and measured?).

System / study	Typed facts	Derived-value certification	Fail-closed soundness	Token-matched causal eval	Gaming evaluation
PCN [arXiv:2509.06902]	Partial (atomic values)	No (declared future work)	Yes (atomic quotes only)	No (no experiments)	No
FinGround [arXiv:2604.23588]	No	Partial (47 templates, ~80% of its second eval set)	No	No	No
VeNRA [arXiv:2603.04663]	No (fact ledger)	No	No	No	No
AuditFlow [arXiv:2606.07031]	No (XBRL filings)	Partial (audits filings, not answers)	No	No	No
FinVerBench [arXiv:2605.29586]	No	Measures verifiers; provides none	No	No	No
XBRL Calculations 1.1 [XBRL International 2023]	Yes	Partial (within-filing summations)	N/A (filing validation)	No	No
FinTagging [arXiv:2505.20650]	No (taxonomy tagging)	No	No	No	No
TOON + generation studies [github; arXiv:2603.03306]	No	No	No	Partial (tokens measured, not matched)	No
SECQUE [arXiv:2504.04596]	No	No	No	Partial (format×token, no verification)	No
TQA-Bench [arXiv:2411.19504]	No	No	No	Partial (multi-scale design)	No
FinQA / ConFinQA [arXiv:2504.25386]	No	No (gold programs)	No	No	No
FinQA [arXiv:2504.25386]	No (proposed)	No (per execution, not certification)	No (theorem + Lean formalization, per proposal)	No (first such design)	No (red-team)
TAT-QA [arXiv:2109.00122]	No	No	No	No	No
CertiFacts (this thesis) [2210.03849, 2105.07624]	No	No	No	No	No

8.2 Reading the matrix

One reading rule first: the FinOKF + CertiFacts row records the commitments of the thesis proposal — designs, theorems to be proved, and experiments to be run — not accomplished results; every cell of that row is prospective. Walking the columns makes the shape of the gap precise. The *typed facts* column is well populated: XBRL itself, VeNRA’s ledger, AuditFlow’s filing environment, and FinTagging’s benchmark show typed numeric representation is established practice. The *derived-value certification* column thins out immediately: FinGround recomputes derived answers but only through 47 hand-written templates covering about 80% of one dataset, with no guarantee outside template coverage; XBRL Calculations 1.1 performs genuinely rounding-aware interval checking, but only for calculation-linkbase summations within a single filing; AuditFlow audits filings, not LLM answers; PoT executes derivations without certifying their operands. The *fail-closed soundness* column has exactly one occupied cell: PCN proves fail-closed certification (its Theorems 5.1 and 5.3) — for atomic quoted values only, with derived values named as future work and, notably, zero experiments in the paper and zero citations as of mid-2026. The *token-matched causal evaluation* column contains only partial credit anywhere: format studies measure token counts and format effects but never hold budgets fixed across representational arms while measuring a verification-relevant outcome. The *gaming evaluation* column is empty: no reviewed work attacks its own verification mechanism and reports certified-but-wrong rates under adversarial pressure. Restated at the level of the thesis’s headline artifact, the gap is the four-way empty combination stated at the top of §8.1: OKF supplies an open container but deliberately no numeric semantics (§5.4); XBRL supplies typed numeric semantics but no answer-level certification (§4.4, §5.1); PCN supplies fail-closed certification but not for derived values (§3.4); and no format study evaluates typing causally at matched token budgets (§6). Open container, typed numeric semantics, derivation certification, causal evaluation: no reviewed work combines them, and that combination is the cell FinOKF + CertiFacts is designed to fill.

Two honest qualifications. First, no single column is unprecedented — the novelty claimed here is the *conjunction*, and the tolerance algebra proposed in this thesis (built on the interval and rounding foundations of §4) is what makes the derivation column reachable at all, since without interval semantics over rounded facts a derived-value checker either rejects nearly everything (exact comparison against rounded inputs) or certifies too much (ad hoc tolerances with no error model). Second, the nearest neighbor is close: the CertiFacts layer is best described as PCN extended from atomic quotation to derivations, inheriting its fail-closed framing and extending its soundness results — a completion of a declared research agenda, not a departure from it. The crowding timeline in this lineage (FinGround, FinVerBench, and AuditFlow appeared within four months of each other in 2026) suggests the conjunction will not remain empty long.

8.3 Open questions

Four questions emerge from this review that the field — and this thesis, in part — must answer. They are labeled Q1–Q4 as open questions of the field, distinct from the thesis’s research questions RQ1–RQ4, which are commitments of the proposal.

Q1: What is the certification coverage ceiling? FinVerBench’s rule-based verifier hit a recall ceiling near 52% [FinVerBench, arXiv:2605.29586]: roughly half of true statements could not be

mechanically verified. What fraction of realistic analyst questions can be expressed as quote-or-derive claims over deterministically constructed fact bundles is unknown, and the fail-closed design converts low coverage into a usability cost rather than a soundness failure — but the cost must be measured, not assumed away. This thesis reports coverage as a first-class metric; the open question is whether coverage can be *raised* without weakening soundness.

Q2: How should typed formats and checkers treat issuer-defined concepts? With roughly one fact in seven carrying an extension tag [FinTagging, arXiv:2505.20650], any certification story that only works over the standard taxonomy silently excludes a seventh of the data. Within-firm derivations over extension facts are checkable; cross-firm comparability of extension concepts is not, absent mapping assumptions nobody has validated. A principled semantics for certified claims over heterogeneous concept vocabularies is open.

Q3: Is there an emission regime where the constraint tax stays below the grounding gain? The constrained-decoding collapse documented in [TOONGen, arXiv:2603.03306] and the 9–14 point exotic-notation penalty [FormatCost, arXiv:2605.29676] show structured emission has real costs that plausibly vary with model scale and schema complexity. Whether plain-JSON claim-graphs, intermittent constraining, or post-hoc repair minimizes the tax — and whether the answer changes across model families — is an empirical question this thesis measures for one design point only.

Q4: After the documentation confound is controlled, how much of the typed-structure effect survives? The semantic-layer results [SemanticLayer, arXiv:2604.25149] leave open the possibility that most claimed structure effects in the literature are documentation effects. The factorial design in G2 answers this for one format and two models; the general question — whether structure has any causal effect on numeric accuracy independent of the prose describing it, and whether that effect interacts with scale — is unresolved and, given how much of the field’s motivation rests on it, surprisingly untested.

9. References

Entries are alphabetized by citation key. Recent arXiv works are listed by short name and arXiv identifier; where a canonical paper title was not independently verified during this review, a descriptive label is used in place of a title. Classic references and specification documents were verified against publisher, arXiv, or standards-body records in July 2026.

- AffineArithmetic — de Figueiredo, L. H., and Stolfi, J. Affine Arithmetic: Concepts and Applications. Numerical Algorithms 37, 147–158 (2004).
- AuditFlow. arXiv:2606.03031 (2026). <https://arxiv.org/abs/2606.03031>
- ConvFinQA. arXiv:2210.03849 (2022). <https://arxiv.org/abs/2210.03849>
- CoT — Wei, J., et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903 (2022). <https://arxiv.org/abs/2201.11903>
- FAITH. arXiv:2508.05201 (2025). <https://arxiv.org/abs/2508.05201>
- FIBO — Financial Industry Business Ontology. EDM Council / Object Management Group. <https://spec.edmcouncil.org/fibo/> (accessed July 2026).
- FinanceBench. arXiv:2311.11944 (2023). <https://arxiv.org/abs/2311.11944>

- FinDER. arXiv:2504.15800 (2025). <https://arxiv.org/abs/2504.15800>
- FinGround. arXiv:2604.23588 (2026; ACL 2026 Industry track). <https://arxiv.org/abs/2604.23588>
- FinQA. arXiv:2109.00122 (2021). <https://arxiv.org/abs/2109.00122>
- FinTagging. arXiv:2505.20650 (2025). <https://arxiv.org/abs/2505.20650>
- FinVerBench. arXiv:2605.29586 (2026). <https://arxiv.org/abs/2605.29586>
- FormatCost — format-cost corroboration study. arXiv:2605.29676 (2026). <https://arxiv.org/abs/2605.29676>
- IntervalAnalysis — Moore, R. E., Kearfott, R. B., and Cloud, M. J. Introduction to Interval Analysis. SIAM (2009).
- Lean4 — de Moura, L., and Ullrich, S. The Lean 4 Theorem Prover and Programming Language. CADE-28, Springer (2021).
- mathlib — The mathlib Community. The Lean mathematical library. arXiv:1910.09336 (2019). <https://arxiv.org/abs/1910.09336>
- OKF — Open Knowledge Format specification v0.1 (draft), okf/SPEC.md. GoogleCloudPlatform/knowledge-catalog, Apache-2.0. <https://github.com/GoogleCloudPlatform/knowledge-catalog> (accessed July 2026).
- PCN — Proof-Carrying Numbers. arXiv:2509.06902 (2025). <https://arxiv.org/abs/2509.06902>
- PoT — Chen, W., et al. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. arXiv:2211.12588 (2022; TMLR 2023). <https://arxiv.org/abs/2211.12588>
- PromptCompression — prompt-compression serving study. arXiv:2604.02985 (2026). <https://arxiv.org/abs/2604.02985>
- SECQUE. arXiv:2504.04596 (2025). <https://arxiv.org/abs/2504.04596>
- SemanticLayer — semantic-layer documentation-confound study. arXiv:2604.25149 (2026). <https://arxiv.org/abs/2604.25149>
- TAT-QA. arXiv:2105.07624 (2021). <https://arxiv.org/abs/2105.07624>
- TOON — Token-Oriented Object Notation. <https://github.com/toon-format/toon> (accessed July 2026).
- TOONGen — TOON generation study (21 models). arXiv:2603.03306 (2026). <https://arxiv.org/abs/2603.03306>
- TQA-Bench. arXiv:2411.19504 (2024). <https://arxiv.org/abs/2411.19504>
- VeNRA. arXiv:2603.04663 (2026). <https://arxiv.org/abs/2603.04663>
- XBRL21 — XBRL 2.1 specification (decimals attribute). XBRL International. <https://specifications.xbrl.org/work-product-index-group-base-spec-base-spec.html> (accessed July 2026).
- XBRLCalc11 — XBRL Calculations 1.1, Recommendation 2023-02-22. XBRL International (2023). <https://www.xbrl.org/Specification/calculation-1.1/REC-2023-02-22/calculation-1.1-REC-2023-02-22.html> (accessed July 2026).